

07 - Testing

Ejercicio 07 — Testing

Foco: testing, casos borde, datos de prueba **Tiempo estimado:** 2 clases [← Volver al README](#)

Objetivo

Que uses la IA para **escribir tests** de un proyecto que hoy **no tiene ninguno**. Y, más importante, que aprendas a pedirle a la IA **casos de prueba que valgan la pena** —no solo el happy path— y a darte cuenta cuando un test "pasa" pero en realidad no prueba nada.

El problema

El proyecto **no tiene test framework** (mirá `package.json`). Todo se prueba a mano con Postman. Eso significa que cada vez que refactorizás (ejercicios 02 y 03), **no tenés red de seguridad**: si rompiste algo, te enterás cuando un endpoint falla en producción.

Hay lógica que **pide tests a gritos**:

- `calcularEdad(fechaNacimiento)` en `alumnos-service.js` → función pura, fácil de testear, **llena de casos borde** (cumpleaños hoy, fecha futura, fecha inválida, `null`).
 - `validarCursoExiste` → lógica de negocio que lanza error.
 - Los endpoints CRUD → tests de integración (status codes correctos).
-

Qué tenés que lograr

1. Elegir y configurar un test runner (el `node:test` nativo de Node es lo más liviano; Jest o Vitest también valen —justificá).
 2. **Tests unitarios** de `calcularEdad` cubriendo casos borde.
 3. **Al menos un test de integración** de un endpoint (que arranque la app o use `supertest`).
 4. Un script `npm test` que los corra.
-

Cómo encarar el prompting

💡 El testing es **EL** lugar donde la IA brilla y donde más fácil te engaña a la vez. Brilla porque genera muchos casos rápido. Te engaña porque genera tests que **siempre pasan** (testean lo que el código hace, no lo que debería hacer).

Prompt de generación de casos (antes del código):

"Para esta función `calcularEdad`, listame todos los casos borde que debería testear, incluyendo entradas inválidas. No escribas los tests todavía, solo la lista de casos con el resultado esperado de cada uno."

Después revisás **vos** la lista y recién ahí pedís el código.

⚠️ **Trampa #1 — el test tautológico:** la IA a veces escribe `expect(calcularEdad(fecha)).toBe(calcularEdad(fecha))` o copia el resultado que devuelve la función actual como "esperado". Eso **no prueba nada**: si la función está mal, el test también. Vos tenés que poner el resultado esperado **a mano**, calculándolo vos.

⚠️ **Trampa #2 — `new Date("texto")`:** en JS, `new Date("caca")` devuelve un `Invalid Date` que **es** un `Date` (`instanceof Date === true`) pero da `NaN` en las cuentas. ¿Qué devuelve `calcularEdad` con eso hoy? Tu test tiene que cubrir ese caso. (Pista: probalo en Node antes de escribir el test.)

⚠️ **Trampa #3 — fecha "hoy":** un test que use `new Date()` (hoy) puede pasar hoy y fallar mañana o el día de tu cumpleaños. Pedile a la IA que **mockee la fecha actual** o use fechas fijas.

💡 **Generación de datos de prueba** (🟢 totalmente habilitado por EFSI): pedile a la IA que te genere un set de alumnos de prueba con fechas variadas para tus tests.

🔍 Verificación del resultado

- ☐ `npm test` corre y los tests pasan.
- ☐ Hay un test que **falla si rompés `calcularEdad` a propósito** (probalo: cambiá un `-` por un `+` en la función y mirá que el test se ponga rojo). Si sigue verde, el test es inútil.
- ☐ Hay un caso para fecha inválida y uno para `null`.
- ☐ El test de fecha **no depende del día en que se corre**.
- ☐ El test de integración verifica el **status code** además del body.
- ☐ Entendés la diferencia entre el test unitario y el de integración y podés explicarla.

😬 Pregunta para el oral: rompé `calcularEdad` adelante mío y mostrame qué test se pone rojo. Si ninguno se rompe, ¿para qué sirve el test?

✅ Entrega

Bitácora + commit con los tests. **Obligatorio:** en la reflexión, contá un caso borde que **vos** pensaste y la IA no había incluido en su lista inicial.